

VPLS on Software Defined Network Using ONOS Controller Based on Raspberry-Pi 3

1st Rizal Cerdas Kurniawan

School of Applied Science

Telkom University

Bandung, Indonesia

rizalcerdaskurniawan@gmail.com

2nd Rohmat Tulloh

School of Applied Science

Telkom University

Bandung, Indonesia

rohmatth@telkomuniversity.ac.id

3rd Indrarini Dyah Irawati

School of Applied Science

Telkom University

Bandung, Indonesia

indrarini@telkomuniversity.ac.id

Abstract—In this paper, Software Defined Network (SDN) technology is applied using the Open Network Operating System (ONOS), which can separate the control plane and the data plane on a network device. ONOS is installed on Raspberry-Pi 3 to minimize the use of Personal Computer (PC) devices on SDN networks. ONOS is also implemented to support Virtual Private LAN Service (VPLS) features that create the Layer-2 network by using the OpenFlow protocol. Software is connected to the host network by connecting to network overlays that are connected to the OpenFlow data plane protocol. VPLS can maximize the existing bandwidth when communication occurs between remote network devices on a local network. We tested voice over Internet Protocol (VoIP) with a variety of background traffic. The test results show that the network performance meets the good level according to the THIPON standard.

Keywords— Software Defined Network, ONOS, Raspberry-Pi 3, VPLS

I. INTRODUCTION

The VoIP market's growth has increased significantly since 2016, so it is necessary to increase effective and cost-efficient VoIP services [1]. The existing network-based infrastructure must meet the increase in service needs. A network infrastructure that vendors almost entirely manage causes increasingly complex networks to be built, so we need a system to manage and implement networks to support diverse needs. Software-Defined Network (SDN) is a new concept in controlling, implementing, and managing a network that supports the needs and innovations in the telecommunications sector that are increasingly growing and complex [2]. Sdn concept is to separate control plane and data plane. SDN can make networks both small scale and large scale capable of being controlled using one centralized controller [3]. Some of the controllers in SDN include POX, RYU, OpenDaylight, and ONOS. Network Operating Systems must meet requirements that demand Scalability, Performance, and Availability to support large-scale networks. To overcome these challenges, ONF introduced the Open Network Operating System (ONOS) [4].

ONOS is a Java-based controller that utilizes the framework's Open Service Gateway (OSGi) initiative to more easily install and update applications [5].

Another concept is also used to gain effectiveness on the existing network, the VPLS. The advantage concept of VPLS is providing Ethernet-based multipoint to multipoint communication via an IP network. It can overcome the spread geographic constraints of sharing Ethernet broadcast domains by connecting virtually.

Several previous studies referenced, among others in [6], the authors analyzed several SDN controllers such as NOX, POX, RYU, Beacon, OpenDaylight, Onix, and ONOS. POX and Maestro controllers still work very well, even though the network has many switch devices. Simultaneously, the ONOS controller has better performance when it also works as an SDN control plane for local area networks (LAN) and data center networks. In research [7], the SDN firewall has been implemented and performed using a POX controller, but this study was only carried out with measurements of Wireshark, ICMP, and iperf with mininet simulation. In research [8], control of open and disaggregated transport networks with ONOS has been conducted, implementation and simulation carried out with the NETCONF protocol to consider establishing connectivity services and connectivity recovery in the event of a failure during data transmission. In other research [9], the authors propose the concepts of SDN combined with Network Function Virtualization (NFV) to improve VPLS network performance. System performance is measured by running the TCP service. Regardless of the expected benefits, some limitations arise include interoperability, security, performance, and scalability.

This study carried out the SDN network's design and implementation using an ONOS controller installed on Raspberry-Pi 3 with the Virtual Private LAN Service (VPLS) application. We build our network consists of 3 OpenFlow switches, 4 Laptops, and 1 Raspberry-Pi 3 as a controller. To find out the performance of the proposed system, we provide VoIP services on the network. System performance is measured based on parameters throughput, delay, jitter, packet loss. The performance must meet the requirements based on the Telecommunications and Internet Protocol Harmonization Over Network (TIPHON) standard [10].

II. OVERVIEW

A. Software-Defined Network

Software-Defined Network (SDN) is a new concept in controlling, implementing, and managing a network that supports the needs and innovations in telecommunications that are increasingly growing and complex. SDN concept is to **separate the control plane and data plane** [11]. SDN can make networks both small scale and large scale capable of being controlled using **one centralized controller** [12-14]. Some controllers in SDN include POX, RYU, OpenDaylight, and ONOS.

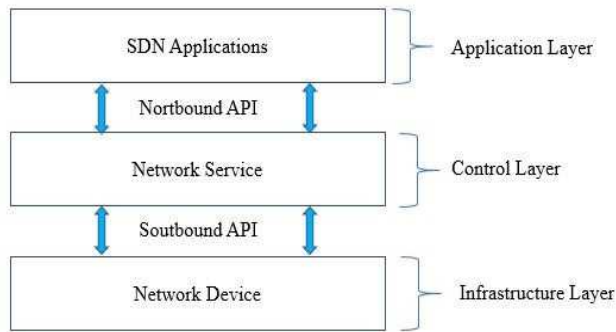


Fig. 1. SDN architecture

Fig. 1 shows SDN Architecture consists of 3 layers.

- **Data Plane:** consists of network elements that can manage Datapath SDN following instructions given via the Control-Data-Plane Interface (CDPI).
- **Control Plane:** Part of a network service as an SDN controller that functions to translate the application layer requirements to the infrastructure layer. This process is carried out by providing appropriate SDN Datapath and providing relevant information required by the Application layer.
- **Application:** located at the top layer, communicating with the system via the Northbound Interface (NBI).

B. Open Network Operating System

ONOS is one of the controllers besides NOX, POX, Ryu, Floodlight, OpenDaylight (ODL), Beacon, and many others, which continue to grow. Each controller from these various vendors has a different approach to implementing the controller's communication process. ONOS is an open-source controller and functions as the central control plane on SDN. ONOS is an open-source project distributed Java-based programming language [15]. With ONOS, we can **manage network elements**, such as **switches and routing**, and run and develop software programs in modularity, providing reliable communication services, high scalability, and expected to have better performance.

C. Virtual Private LAN Service

VPLS is a **multipoint VPN layer 2** that allows many regions to be connected in the same single **bridge domain over the Internet Protocol (IP) network**. All client regions in the VPLS instance can be as if they are on the **same LAN network** even though it is geographically separated [16]. VPLS uses an **Ethernet interface to the client**. VPLS services are provided for operators, service providers, and large companies that require the availability of high-performance

Ethernet services and Quality of Service (QoS) guarantees [17].

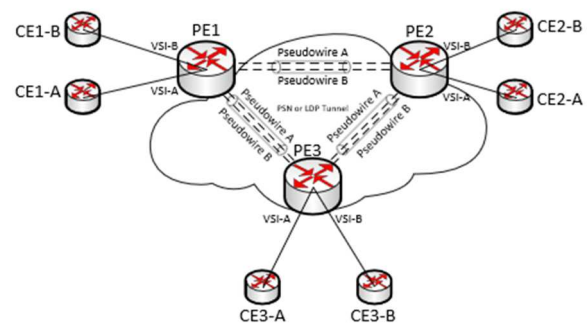


Fig. 2. VPLS Concept

Fig. 2 shows some of the advantages of being a VPLS are better than the previous tunneling technology, namely that VPLS has **pseudowires as virtual circuits**. VPLS also has **auto-discovery and auto-configuration**, allowing several devices on the VPLS network to recognize each other and **build pseudowires for newly recognized devices**. VPLS consists of Virtual Switched Instance (VSI) or Virtual Forwarding Instance, which can define members of the VPLS domain and **resemble** virtual switches on a PE router. VPLS allows ethernet interfaces that have the same virtual LAN to be connected with many PE devices. For example, user A in the VPLS domain consists of an ethernet interface connected to user A on the CE router in a different place. VSI can **remotely MAC addresses and ensures the VPLS domain is free from looping**. VSI has the functions of MAC address management, flooding, and data forwarding [18].

D. Raspberry-Pi 3

This study using a mini-computer as a controller. A third-generation Raspberry-Pi type mini-computer that uses an **ARM System-on-chip (SoC)**. Raspberry 3 was chosen because it is equipped with **4 USB 2 ports and a 100 Base Ethernet port** to connect with an open flow switch. The network configuration in this study requires a **minimum of 3 ethernet ports for three switches**. Consequently, **two USB ports are converted to ethernet ports using a USB to ethernet converter**. This mini-computer also provides 1GB of RAM and a 1.2GHz 64-bit ARMv8 quad-core CPU. This processor and RAM are handy when performing control plane and network management functions. Several other functions are available such as Bluetooth 4.1 support, 80.11n wireless LAN, and support Bluetooth Low Energy (BLE)

III. SYSTEM IMPLEMENTATION

A. Network Topology

Three applications are active on the ONOS **controller**, that is, **OpenFlow, Forwarding, and VPLS**. Configuring VPLS on the ONOS terminal creates two VPLS IDs, namely VPLS1 and VPLS2. Hosts with different VPLS IDs cannot communicate with each other. On the other hand, hosts that have the same VPLS ID can communicate with each other. This type of service uses **VoIP using Asterisk, Zoiper, and iperf software** as background traffic generators. PC 1 as a VoIP server and three other PCs as a client. QoS measurement parameters consist of throughput, delay, packet loss, and jitter using Wireshark software.

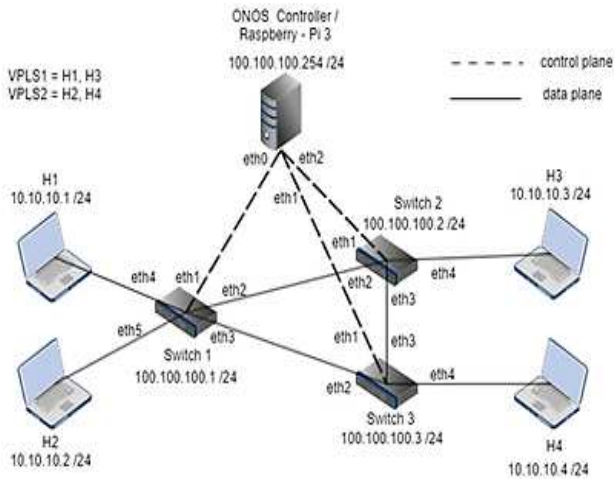


Fig. 3. Network topology design

Fig. 3 shows some devices used on this network, namely 1 Raspberry-Pi 3 as an ONOS controller, 2 USB-To-Ethernet converters, 3 OpenFlow switches, 10 UTP cables, and 4 PCs.

B. Bridge Configuration

Raspberry-Pi 3, which is used as a controller device, only has one ethernet interface [19], for that it is done by adding devices, namely 2 USB-to-Ethernet converters on Raspberry-Pi 3 so that it **can connect directly to the Raspberry-Pi 3 controller device** with a switch OpenFlow on SDN networks.

```
auto br0
iface br0 inet static
    address 100.100.100.254
    netmask 255.255.255.0
    network 100.100.100.0
    bridge_ports eth0 eth1 eth2
```

Fig. 4. Bridge Configuration

In the implementation process so that **every OpenFlow switch connected to the controller can communicate on the same network**, a bridge configuration is required on the controller interface [20]. Fig. 4 shows the addition of a new bridge interface, br0, with a static IP Address configuration with three port interfaces being bridged, namely eth0, eth1, eth2.

C. OpenFlow Switch Configuration

On an SDN network, not all network devices can be directly recognized by the controller, only devices that support the OpenFlow protocol can be recognized [21]. Therefore in this paper, using the **OpenFlow switch with a Mikrotik device**. At this stage, the OpenFlow switch configuration is performed, several configurations are performed, namely **IP Address, bridge, OpenFlow as the control plane, and the data plane**.

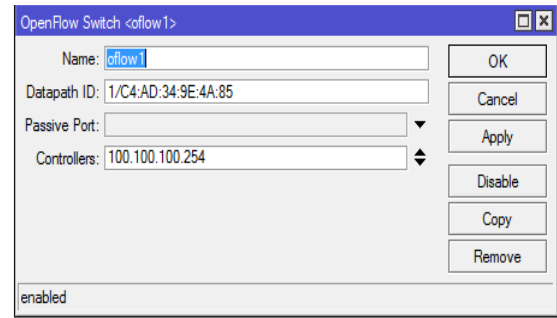


Fig. 5. New OpenFlow Interface

Fig. 5 shows the configuration by adding the controller IP Address so that the controller can recognize the switch.

Interface	Switch	Port	Tx Bytes	Tx Packets	Tx Drops	Rx Bytes	Rx Packets	Rx Drops
ether2	oflow1	1	0B	0B	0B	0B	0B	0B
ether3	oflow1	2	0B	0B	0B	0B	0B	0B
ether4	oflow1	3	0B	0B	0B	0B	0B	0B
ether5	oflow1	4	0B	0B	0B	0B	0B	0B

Fig. 6. Port Forwarding Configuration

Fig. 6 shows the OpenFlow data plane configuration port with the specifications:

- Ether2: connected to switch2
- Ether3: connected to switch3
- Ether4: connected with PC1
- Ether5: connected with PC2

D. VPLS Configuration

In this paper, a VPLS configuration is performed by creating **2 VPLS IDs with the ID names**, namely VPLS1 and VPLS2. Each VPLS ID consists of 2 PCs. VPLS1 consisting of PC1 and PC3, and VPLS2 is consisting of PC2 and PC4.



Fig. 7. VPLS ID Addition

Fig. 7 shows VPLS ID is added with the names VPLS1 and VPLS2.

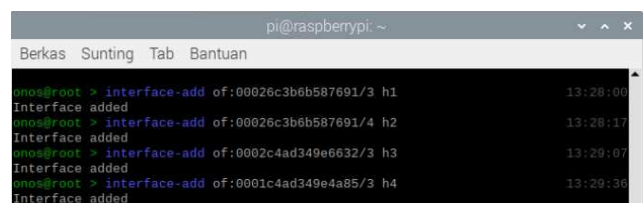


Fig. 8. Provide ID On The Host Interface

Fig. 8 shows ID is added to each host's interface so that VPLS can recognize it.

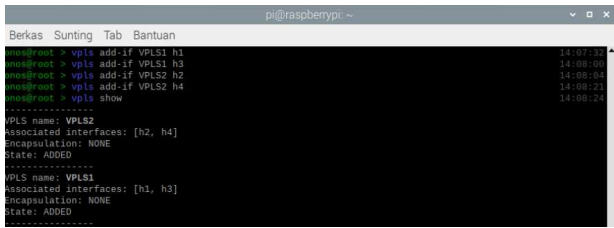


Fig. 9. VPLS Group

Fig. 9 shows the grouping of Hosts on VPLS ID. PC 1 and PC 3 into VPLS1 while PC 2 and PC 4 into VPLS2.

E. System Specification

• Controller Specification

For the controller using Raspberry-pi 3 with specifications as shown in table 1

TABLE 1. Raspberry-Pi 3 Specification

Type	Specification
Central Processing Unit	4x ARM Cortex-A53, 1.2 GHz
RAM	1 GB
Network	10/100 Ethernet, 2.4 GHz 802.11n WLAN
Operating System	Raspbian 4.19

Fig. 10 shows a schematic circuit of the Raspberry-Pi 3 used. In this study, 2 USB-to-Ethernet converters are used to connect the Raspberry-Pi 3 with OpenFlow switches, namely Switch 2 and Switch 3 and 1 Ethernet port Switch 1. All ports will be connected to the switch using a UTP cable. To connect all OpenFlow switches with this controller, a bridge configuration is done on the controller side. Raspberry-pi configuration as a controller

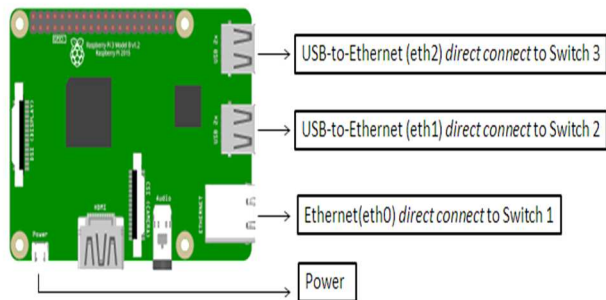


Fig. 10. Raspberry-pi 3 configuration as a controller

• Switch Specification

For the forwarding plane function using a Mikrotik Router with Openflow installed with the specifications as in table 2.

TABLE 2. Mikrotik Specification

Type	Specification
Operating System	RouterOS
Number of LAN Port	5
RAM	128 MB
Routerboard	RB-951Ui-2HnD

The next step is to configure OpenFlow on the switch so that each switch connected to the Raspberry-Pi 3 controller can detect the ONOS topology, which means that ONOS as a

controller can control the SDN network through these switches. The next step is to connect each switch to the PC according to the topology, then configure the IP address on each PC as if it were in a local network with the same network prefix.

IV. RESULT AND ANALYSIS

A. Controller Test

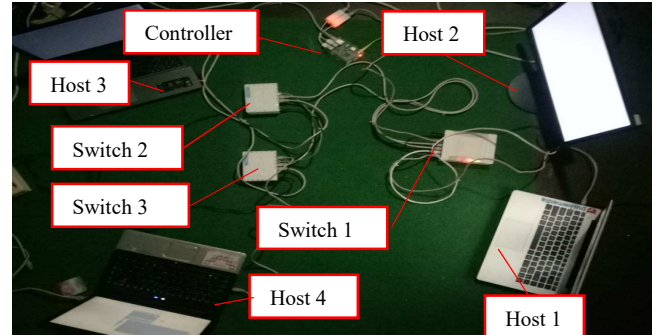


Fig. 11. Network topology implementation

Fig. 11 shows the controller side's test, where the controller already recognizes all devices on the SDN network. In this test, the controller can recognize all available devices according to the implementation topology, where there are three switches and 4 PCs according to topology planning.

B. Connectivity Test

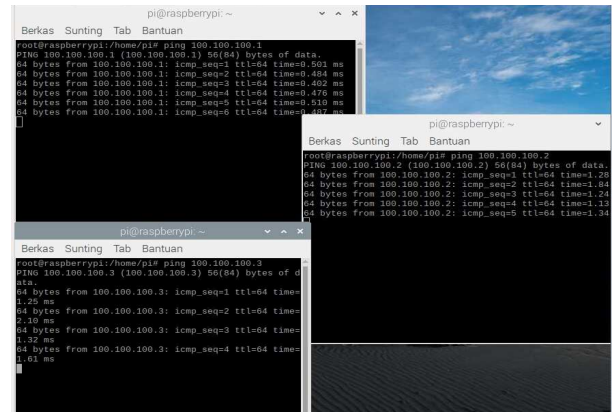


Fig. 12. Connectivity Test Via Controller

Fig. 12 shows network connectivity testing to find out all switch devices connected to the controller can communicate adequately. Fig. 12. also shows that it has successfully performed PING with a packet value of 64 Bytes to 3 switches connected directly to the controller from the controller side.

C. Virtual Private LAN Service Test

This VPLS test is done by doing PING between hosts with the same VPLS ID or hosts with different VPLS IDs. If the VPLS configuration has not been done automatically, all hosts can communicate.

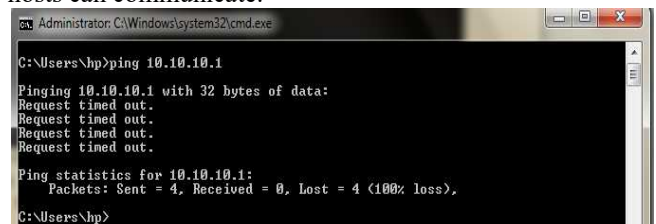


Fig. 13. Host Connectivity with Different VPLS ID

Fig. 13 shows that PING between h2 and h1 generates a Request Time Out because h2 and h1 have different VPLS ID.

```

Administrator: C:\Windows\system32\cmd.exe
C:\Users\hnp>ping 10.10.10.4

Pinging 10.10.10.4 with 32 bytes of data:
Reply from 10.10.10.4: bytes=32 time=16ms TTL=128
Reply from 10.10.10.4: bytes=32 time=2ms TTL=128
Reply from 10.10.10.4: bytes=32 time<1ms TTL=128
Reply from 10.10.10.4: bytes=32 time<1ms TTL=128

Ping statistics for 10.10.10.4:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 16ms, Average = 40ms
C:\Users\hnp>

```

Fig. 14. Host Connectivity with The Same VPLS ID

Fig. 14 shows the PING between h2 and h4 resulting in a TTL Reply, which means that h2 and h4 can communicate because h2 and h4 are included in the same VPLS ID.

D. Performance Test (Voice Over Internet Protocol)

- Throughput

This part will show the measurement of throughput to determine the network's actual speed when sending data. Measurements are made by making calls between h1 and h3 that are already connected and can communicate through VPLS services on SDN.

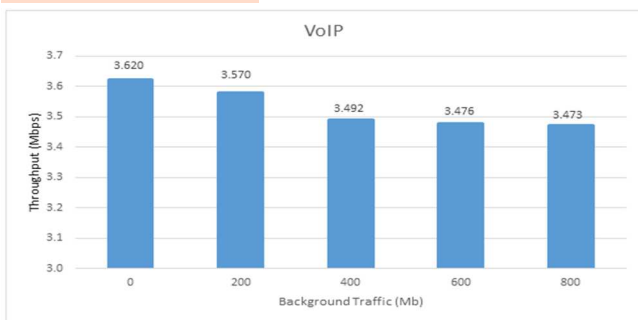


Fig. 15. Throughput Performance

Fig. 15 shows that the throughput generated on this measurement is still following the TIPPHON standard, where the value of throughput without background traffic reaches 3.62 Mbps and the highest throughput value on background traffic reaches 3.473 Mbps. The average overall throughput is 3,528 Mbps.

- Delay

In this part, we take measurements of the delay by calculating the average arrival time of the first or previous packet reduced by the second or after packet's arrival time during the VoIP communication process using Zoiper.

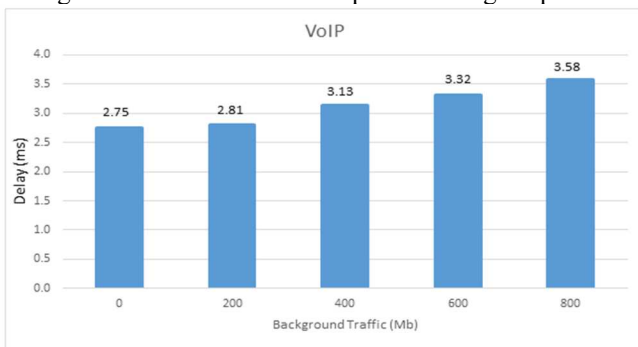


Fig. 16. Delay Performance

Fig. 16 shows the delay from measurements without background traffic valued at 2.75 ms and measurements with the highest background traffic worth 3.58 ms. These results indicate that the delay shown on this measurement is still following the TIPPHON standardization. The average of these overall delay measurements is 3.11 ms.

- Jitter

In this part, jitter is measured by reducing the first or previous packet delay time with the second or subsequent packet delay time.

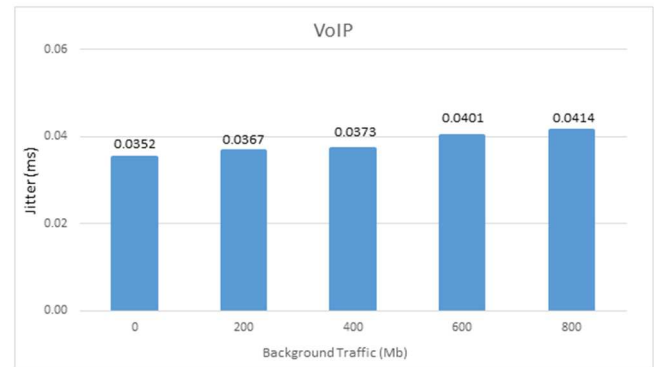


Fig. 17. Jitter Performance

Fig. 17 shows that the value of jitter in this measurement is still following TIPPHON standardization. In this measurement, the higher the value of background traffic, the jitter's value will also increase. The jitter's value without background traffic shows a value of 0.0352 ms, while jitter's value on the highest background traffic is worth 0.0414 ms. The average of these jitter measurements is 0.0381 ms.

- Packet Loss

In this part, the packet loss is measured by counting the number of packets sent minus the number of packets received then multiplied by 100%.

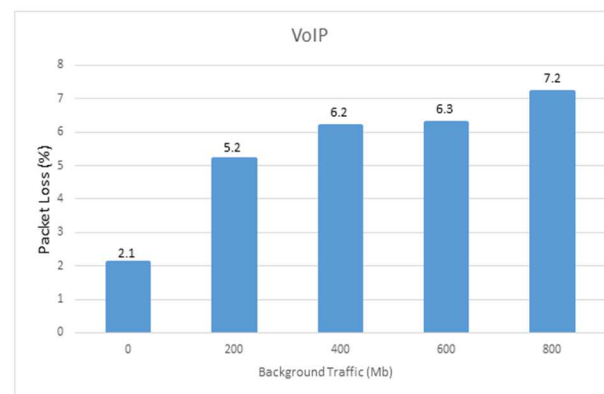


Fig. 18. Packet Loss Performance

Fig. 18 shows that the value of packet loss from this measurement is still following TIPPHON standardization, wherein this measurement, the packet loss value in the measurement without background traffic is 2.1%, and the packet loss value in the highest background traffic measurement is 7.2%.

TABLE 3. Performance Conclusion

N o	Quality of Service	Without Background Traffic	Background Traffic Max	Index without Background Traffic	Index Background Traffic max
1	Throughput	3,62 Mbps	3,47Mbps	Excellent	Excellent
2	Delay	2,75 ms	3,58 ms	Perfect	Perfect
3	Jitter	0,0352 ms	0,0414 ms	Good	Good
4	Packet Loss	2,1%	7,2%	Perfect	Good

Based on Table 3. the results of QoS measurements that have been carried out using VoIP services on this network implementation have good QoS performance following TIPHON standardization.

V. CONCLUSION

We build VPLS on the SDN network that separates the control plane from the data plane. We use ONOS as a controller that is applied to Raspberry-Pi 3. The performance results of VPLS implementation on SDN networks using the ONOS controller applied to the Raspberry-Pi 3 show that the four QoS parameters' values still follow a good TIPHON standard. In this research, a comparison of the results of QoS measurements with 200Mb, 400Mb, 600Mb, 800Mb background traffic is used to determine the network performance limits on traffic loads. The result is the measurement with the highest traffic load, and the QoS value still shows a good value according to TIPHON standards. VPLS implementation can be run on SDN networks using the ONOS controller to share private networks between hosts.

REFERENCES

- [1] A. Bhutani, P. Wadhvani, "Voice over Internet Protocol (VoIP) Market Size By Type," April 2019. [Online]. [Accessed 26 January 2021]. <https://www.gminsights.com/industry-analysis/voice-over-internet-protocol-voip-market>.
- [2] M. Liyanage, A. Gurtov, M. Ylianttila, "Software-Defined Networking Concepts," *Software-Defined Mobile Network (SDMN)*, Wiley Telecom, 2015, 21-44.
- [3] V. Monita, I. D. Irawati, R. Tulloh, "Comparison of routing protocol performance on multimedia services on software defined network," *Bulletin of Electrical Engineering and Informatics*, vol. 9, no. 4, pp. 1612-1619, 2020.
- [4] T. Alharbi, M. Portmann, "The (In)Security of Virtualization in Software Defined Networks," *IEEE Access*, vol. 7, pp. 66584-66594, 2019.
- [5] A. H. Eljack, A. Hassan, H. H. Elamin, "Performance Analysis of ONOS and Floodlight SDN Controllers based on TCP and UDP Traffic," in *2019 International Conference on Computer, Control, Electrical, and Electronics Engineering (ICCCCEE)*, Sudan, 2019.
- [6] A. A. Semenovykh, O. R. Laponina, "Comparative analysis of SDN controllers," *International Journal of Open Information Technologies*, vol. 6, no. 7, 2018.
- [7] W. M. Othman, H. Chen, A. Al-moalimi, A. N. Hadi, "Implementation and performance analysis of SDN firewall on POX controller," in *2017 IEEE 9th International Conference on Communication Software and Networks (ICCSN)*, China, 2017.
- [8] A. Giorgetti, A. Sgambelluri, R. Casellas, "Control of open and disaggregated transport networks using the Open Network Operating System (ONOS)," *IEEE/OSA Journal of Optical Communications and Networking*, vol. 12, no. 2, pp. A171-A181, 2019.
- [9] M. Liyanage, M. Ylianttila, A. Gurtov, "Software Defined VPLS Architectures: Opportunities and Challenges," 2017 IEEE 28th Annual International Symposium on Personal, Indoor, and Mobile Radio Communications (PIMRC 2017)At: Montreal, Canada, DOI: 10.1109/PIMRC.2017.8292519
- [10] Telecommunications and Internet Protocol Harmonization Over Networks (TIPHON), "Tr 101 329," Etsi, vol. 1, no. General aspects of Quality of Service (QoS), pp. 1-37, 1999.
- [11] A. Zahmatkesh, T. Kunz, "Software defined multihop wireless networks: Promises and Challenges," *Journal of Communications and Networks*, vol. 19, no. 6, 2017.
- [12] Y. Zhou, B. Ramamurthy, B. Guo, "Supporting dynamic bandwidth adjustment based on virtual transport link in software-defined IP over optical networks," *IEEE/OSA Journal of Optical Communications and Networking*, vol. 10, no. 3, pp. 125-137, 2018.
- [13] R. Tulloh, H. Tussyadiah, R. W. Hutabri and R. M. Negara, "Load distribution analysis on bipartite topology using floodlight controller," *Journal of Theoretical and Applied Information Technology*, vol. 96, no. 5, pp. 1238-1252, 2018.
- [14] R. Tulloh, J. G. Amri Ginting, A. Mulyana, and M. Lutfi, "Performance Comparison of File Transfer Protocol Service between Link State and Distance Vector Routing Protocol in Software Defined Network," *IOP Conference Series: Materials Science and Engineering*, vol. 982, no. 1, 2020.
- [15] A. S. Muqaddas, P. Giaccone, A. Bianco, "Inter-Controller Traffic to Support Consistency in ONOS Clusters," *IEEE Transactions on Network and Service Management*, vol. 14, no. 4, pp. 1018-1031, 2017.
- [16] C. Fancy, L. M. M. Thanveer, "An evaluation of alternative protocols-based Virtual Private LAN Service (VPLS)," in *2017 International Conference on IoT and Application (ICIOT)*, India, 2017.
- [17] M. Liyanage, M. Ylianttila, A. Gurtov, "Secure Hierarchical VPLS Architecture for Provider Provisioned Networks," *IEEE Access*, vol. 3, pp. 967-984, 2015.
- [18] M. Barreiros, P. Lundqvist, "The VPLS Case Study," *QOS-Enabled Networks: Tools and Foundations*, Wiley Telecom, 2015, 163-191.
- [19] R. Heradio, J. Chacon, H. Vargas, "Open-Source Hardware in Education: A Systematic Mapping Study," *IEEE Access*, vol. 6, pp. 72094-72103, 2018.
- [20] K. Lee, B. Kwon, J. Kang, "Optimal Flow Rate Control for SDN-Based Naval Systems," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 53, no. 6, pp. 2690-2705, 2017.
- [21] M. Yang, H. Rastegarfar, I. B. Djordjevic, "Physical-layer adaptive resource allocation in software-defined data center networks," *IEEE/OSA Journal of Optical Communications and Networking* vol. 10, no. 12, pp. 1015-1026, 2018.